Security Assessment

Item 6 Internal Report

July 2026

Prepared for ether.fi

Disclaimer

This document is intended for **internal** use only.

Initial Commit Hash	Final Commit Hash
edba2b91	ac86bd70eb

Protocol Scope

src/data-provider/EtherFiDataProvider.sol

src/across/AcrossSwapModule.sol
src/enso/EnsoSwapModule.sol
src/libraries/OwnershipBridgeMessageLib.sol
src/ownership-bridge/OwnershipBridgeReceiver.sol
src/ownership-bridge/OwnershipBridgeSender.sol
src/safe/EtherFiSafe.sol
src/safe/EtherFiSafeBase.sol
src/safe/EtherFiSafeCore.sol
src/safe/EtherFiSafeFactory.sol
src/safe/MultiSig.sol
src/safe/OwnerBridgePublisher.sol
src/safe/RecoveryManager.sol
src/top-up/TopUp.sol
src/top-up/TopUpFactory.sol
src/top-up/TopUpV2.sol
src/trading-safe/TradingLens.sol
src/trading-safe/TradingOwnerBridgeReceiver.sol
src/trading-safe/TradingSafe.sol
src/trading-safe/TradingSafeFactory.sol
src/utils/EtherFiDeployer.sol

Medium Severity Issues

M-01: No on-chain minOut/recipient validation for same-chain swaps

Severity: Medium	Impact: Medium	Likelihood: Low
Files: src/enso/EnsoSwapModule.sol	Status: Fixed	

Description:

Neither module decodes the `swapData` it forwards to the swap target, and neither validates the swap's result on-chain. On the Enso path and the Across origin-swap branch, the `Order` fields `minOut`, `recipient`, `dstToken`, and `dstChainId` are carried only for the signed intent and events but they are never enforced. All slippage protection, the output token, and the destination live entirely inside the backend-built calldata that the owners blind-sign.

Several scenarios may be detrimental to users. A compromised or buggy backend can produce call data with:

1. Zero effective slippage protection
2. A different recipient than the one signed
3. The wrong output token

For same-chain swaps, the lack of validation post-swap that the swap calldata performed the intended action specified in the signed order may lead to direct loss of funds.

Recommendations:

We recognize that ensuring all necessary inputs are upheld is difficult for cross-chain swaps which will result in fund delivery in a non-atomic way. Therefore, we recommend adding simple before/after token balance checks for the specified recipient for same-chain swaps only.

Customer Response:

Fixed in commit 3239d4a

Fix Review:

Fixed

M-02: Stale/Replayable LZ Ownership-Bridge Messages Can Overwrite Live TradingSafe Config

Severity: Medium	Impact: High	Likelihood: Low
Files: src/safe/EtherFiSafe.sol	Status: Fixed	

Description:

`configureOwners`, `setThreshold`, `recoverSafe`, and `cancelRecovery` on the OP-side `EtherFiSafe` each publish a LayerZero envelope via `OwnershipBridgeSender`, which `OwnershipBridgeReceiver._lzReceive` applies unconditionally to the corresponding mainnet `TradingSafe` (`applyBridgeConfigureOwners`/`applyBridgeSetThreshold`/`applyBridgeRecover`/`applyBridgeCancelRecovery` in `TradingOwnerBridgeReceiver.sol`) as long as the call comes from `BRIDGE_RECEIVER`. Per the receiver's own docstring, there's no application-level nonce, sequence number, or staleness check — it relies entirely on "LayerZero v2's default ordered-delivery + GUID-based replay protection," and the `applyBridge*` functions on `TradingSafe` apply whatever payload they're handed with no check against the safe's current state or the message's age.

If an envelope fails to land on mainnet — insufficient executor gas, the bridge paused, a transient threshold/owner-count mismatch, or any other delivery hiccup — it isn't discarded. LayerZero stores it as a retryable failure that stays valid and executable by anyone, indefinitely, with no expiry. That creates two concrete problems once the source safe has since moved on:

1. **Stale-state replay:** an old `configureOwners`/`setThreshold`/`recoverSafe`/`cancelRecovery` message that got stuck can later be replayed (by anyone, at any point in the future) and will silently overwrite the TradingSafe's current, correctly-configured owners/threshold/recovery state with whatever was signed at the time of the original, now-superseded call.

2. **Compromised/rotated-signer replay:** because the message was authorized off-chain at signing time on OP and carries no freshness or revocation check, a `configureOwners` (or `recoverSafe`) message signed before a malicious or offboarded owner was rotated out

remains fully valid on mainnet and can still be delivered after the rotation — letting a since-removed multisig configuration take effect on the TradingSafe.

Both scenarios let historical, superseded owner-control state override the safe's live configuration on mainnet without any additional authorization, which is a meaningful privilege-escalation / config-integrity risk for a contract that gates fund movement.

Recommendations:

One approach could be to drop the automatic cross-chain sync entirely and require the safe owner to manually configure the TradingSafe's ownership independently on each chain. It's less convenient, but it removes this entire class of stale/replay risk in favor of a predictable, self-contained model.

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

M-03: applyBridgeRecover Bypasses isRecoveryEnabled on TradingSafe

Severity: Medium	Impact: High	Likelihood: Low
Files: src/trading-safe/TradingSafe.sol	Status: Fixed	

Description:

`RecoveryManager.recoverSafe()` on the source-chain safe explicitly checks `isRecoveryEnabled` before staging an incoming owner. `TradingOwnerBridgeReceiver.applyBridgeRecover()` — the destination-chain path invoked by `OwnershipBridgeReceiver` after a `recoverSafe` publish — skips that check and calls `_setIncomingOwner` directly. This means a TradingSafe's own local `isRecoveryEnabled = false` setting, which its owners may have deliberately toggled off as a security posture (e.g. after suspecting compromise, or simply not wanting recovery active on that chain), provides no protection against a bridged recovery: any `recoverSafe` published from the OP-side safe still lands and starts the incoming-owner timelock on the mainnet TradingSafe regardless. This defeats the purpose of `isRecoveryEnabled` as a chain-local control and is compounded by the stale-message-replay issue already flagged — a defensive local disable can't stop a recovery that's mid-flight or gets retried later.

Recommendations:

Have `applyBridgeRecover` respect the TradingSafe's own `isRecoveryEnabled` flag before staging the incoming owner. Don't implement this as a `revert` on disabled recovery, though — that would make the LZ message fail delivery and become a stuck, indefinitely-retryable payload (the exact replay risk from the previous finding), so a legitimate future re-enable of recovery could cause a stale recovery to fire unexpectedly. Instead, treat a disabled-recovery TradingSafe as a clean no-op: consume the message successfully (so LZ marks it delivered and it can't be retried) but skip applying `_setIncomingOwner`, mirroring the pattern `OwnershipBridgeReceiver` already uses for the "TradingSafe not yet deployed" case (emit a deferred/skipped event and return cleanly rather than reverting).

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

M-04: Recover/CancelRecovery Message Reordering Can Resurrect a Cancelled Recovery

Severity: Medium	Impact: High	Likelihood: Low
Files: src/trading-safe/TradingSafe.sol	Status: Fixed	

Description:

A plausible sequence: a TradingSafe's OP-side recovery signers get compromised (or a legitimate-but-erroneous recovery is triggered), and someone calls `recoverSafe(attacker, ...)` on the OP safe — this sets the incoming owner locally and publishes a `Recover` envelope (M1) toward mainnet. The safe's regular owners spot it within the recovery timelock window and immediately call `cancelRecovery()` on OP, which clears the incoming owner locally and publishes a `CancelRecovery` envelope (M2). Both messages are independent LZ sends with no relationship enforced between them. If M1 experiences any delay relative to M2 — a slow/underfunded executor job, a transient failure requiring retry, ordinary network variance — M2 can land on the mainnet `TradingSafe` first (a harmless no-op, since no incoming owner is set there yet), and M1 can land afterward, setting the incoming owner and starting the mainnet recovery timelock. The result: the source-of-truth OP safe has the recovery cancelled, but the mainnet TradingSafe now has an active, ticking recovery that the owners already believed they'd stopped — and they may not notice until it's close to executing, since cancelling on OP gave no on-chain guarantee about mainnet ordering. This is a sharper instance of the same root cause as the stale-replay finding (no per-safe ordering/version enforcement across bridged operations), but worth flagging separately because it doesn't require an old message resurfacing much later — just two messages sent close together arriving out of order.

Recommendations:

Consider dropping the cross-chain owner syncing or alternatively implement a mechanism for tracking message order that doesn’t cause DOS on the system in case a messages in the pipeline becomes un-executable

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

M-05: Zero Withdrawal Delay Permanently Bricks Swaps and Strands Funds in Across/EnsoSwapModule

Severity: Medium	Impact: High	Likelihood: Low
Files: src/enso/EnsoSwapModule.sol	Status: Fixed	

Description:

When the CashModule's `withdrawalDelay` is configured to `0`, `requestWithdrawalByModule(safe, order.srcToken, order.srcAmount)` doesn't just place a hold — it synchronously pulls the full amount out of the safe and into the calling module's own balance via `_processWithdrawal`, then clears `pendingWithdrawalRequest` before returning. `_storeAndDispatch` in `EnsoSwapModule` doesn't check for this — it only branches on `address(cashModule) != address(0)`, so it leaves the swap sitting in `$.swaps[safe]` expecting a later `executeSwap(safe)` call to complete it, exactly as if a real delay were in effect. But by the time that happens, the tokens are no longer in the safe:

- `executeSwap` reverts at `cashModule.cancelWithdrawalByModule(safe)` — that function checks `pendingWithdrawalRequest.tokens.length == 0` and reverts `WithdrawalDoesNotExist()`, since the request was already resolved and deleted back in `_storeAndDispatch`.
- `cancelSwap` hits the identical `cancelWithdrawalByModule` call and reverts the same way.
- `cancelExpiredSwap` — same call, same revert.

So none of the three exit paths can ever succeed once a swap is requested under a 0-delay CashModule: the stored swap is permanently stuck (`OrderAlreadyActive` then blocks any future `requestSwap` for that safe, forever), and the actual tokens are sitting in the swap module contract's own balance — not in the safe, not swapped, and unreachable through any function in either module. This is a full, permanent DoS on the swap module for that safe plus genuinely stranded user funds.

Recommendations:

Check if delay is set to 0 and revert the flow to ensure module works only under expected conditions

Customer Response:

Fixed in **#223** — commit `a3a7d65`

Fix Review:

Fixed

M-06: Chain-local owner addresses can freeze or compromise destination TradingSafes

Severity: Medium	Impact: High	Likelihood: Low
Files: src/trading-safe/TradingOwnerBridgeReceiver.sol	Status: Fixed	

Description:

The ownership bridge represents owners solely as 20-byte addresses and applies those same addresses to the destination TradingSafe. This is unsafe because the Safe explicitly supports ERC-1271 contract owners, while contract code and state are chain-local.

`SignatureUtils` dynamically decides how to authenticate an owner using the code deployed at that address on the current chain:

- If the address has code, it trusts that local contract's `isValidSignature`.
- If the address has no code, it treats the address as an EOA and attempts ECDSA recovery.

Consequently, a contract owner valid on the source chain does not necessarily represent the same security principal on the destination chain:

1. **Destination address is codeless:** A source-chain ERC-1271 owner becomes an apparent EOA on the destination. No corresponding private key normally exists for a contract-created address, so owner-authorized destination actions cannot satisfy the threshold. A sole contract owner or a threshold dependent on that owner can freeze all destination assets.
2. **Destination address contains different code or state:** Contracts deployed with `CREATE` can share an address across chains while using different creation code. Proxies can also exist at the same address while pointing to different implementations or holding different owners in storage. An attacker controlling the destination-side contract can return the ERC-1271 magic value for arbitrary digests and thereby satisfy the TradingSafe's owner threshold.

3. **Recovery inherits the same problem:** `RecoverData.newOwner` is also bridged as a raw address. A valid source-chain recovery to a contract owner can therefore immediately lock or compromise the destination Safe after the recovery timestamp.

This can lead to complete loss of control over, or theft of, every asset held by an affected TradingSafe.

Recommendations:

Consider dropping the cross-chain ownership syncing mechanism

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

M-07: Swap target (ensoRouter/spokePool/periphery) not bound in the signed digest

Severity: Medium	Impact: High	Likelihood: Low
Files: /src/enso/EnsoSwapModule.sol	Status: Fixed	

Description:

The swap modules split a swap into a signed swap request and a permissionless swap execution. On dispatch, the module has the safe `approve(target, srcAmount)` and forwards the owner-supplied `swapData` to that target (`ensoRouter`) for Enso, or the `spokePool` / `multicallHandler` / `peripheryAddress` for Across.

The signed digest does not cover the target. `_requestDigest` binds `order` and `swapData` (plus `depositArgs`/`message` on Across) but not the target address, which is an admin-set value that `executeSwap` reads live from storage. An admin can change the target between the time the swap is requested and executed, which can lead to theft of funds up to the `srcAmount` if the new target is malicious.

Recommendations:

Pin the target for the lifetime of an in-flight swap:

1. Include the target in `_requestDigest` so the signature commits to it, and store it in `StoredSwap` at request time.
2. Have `executeSwap` dispatch against the stored snapshot, not the live storage value.

Customer Response:

Fixed in **#223** — commit `ac86bd7`

Fix Review:

Fixed

Low Severity Issues

L-01: No way to unset periphery after initially setting

Severity: Low	Impact: Low	Likelihood: Low
Files: src/across/AcrossSwapModule.sol	Status: Fixed	

Description:

In `AcrossSwapModule.sol`, the `periphery` address may be set, indicating that origin-swaps are allowed through Across's `anyToBridgeable` periphery contract. However, since the function to set the `periphery` contract guards against `address(0)`, it is not possible to unset a previously set `periphery`.

This means that if the periphery contract is compromised or its execution is unwanted, there is no ability to disable this path. The best option currently is to set to another dead address like `address(1)` however this is harder to understand.

Recommendations:

Allow setting `periphery` to `address(0)` to align with comment `Zero means origin-swaps are not enabled here.`

Customer Response:

Fixed in **#223** — commit `9903ae2`

Fix Review:

Fixed

L-02: orderDeadline that is closer than now + withdrawalDelay will not be executable

Severity: Low	Impact: Low	Likelihood: Low
Files: src/enso/EnsoSwapModule.sol	Status: Fixed	

Description:

`requestSwap` only checks `order.deadline > block.timestamp`, not that the deadline leaves room for the withdrawal delay. `executeSwap` needs both `block.timestamp >= finalizeTime` (= request time + `withdrawalDelay`) and `block.timestamp <= order.deadline`.

If `withdrawalDelay > deadline - now` at request time, the swap can never execute.

Recommendations:

Reject an order whose deadline expires before the withdrawal delay passes.

Customer Response:

Fixed in **#223** — commit `8aa1533`

Fix Review:

Fixed

L-03: The operational deployment role can initialize counterfactual TradingSafes under attacker control

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/trading-safe/TradingSafeFactory.sol	Status: Acknowledged	

Description:

`deployTradingSafe` derives the canonical TradingSafe address solely from `sourceSafe`, but allows any holder of `TRADING_SAFE_FACTORY_ADMIN_ROLE` to choose the initial owners, threshold, modules, and module setup data.

The factory does not require an authorization from the source Safe's current owners.

A compromised role holder can monitor deterministic TradingSafe addresses and front-run legitimate deployment by deploying the Safe with attacker-controlled owners and a usable whitelisted module. This is particularly dangerous because deterministic addresses may already hold assets before code deployment due to cross-chain transfers, misrouted deposits, or counterfactual funding.

Recommendations:

Only allow deployment of trading safes following a cross-chain message from the source chain safe attesting to the critical setup data that should be stored in `OwnershipBridgeReceiver`.

Customer Response:

Acknowledged - *"Acknowledged as an explicit trust assumption. TRADING_SAFE_FACTORY_ADMIN_ROLE is a privileged deployment authority responsible for the TradingSafe's initial configuration. Compromise of this role is outside the contract's trust model."*

Fix Review:

Acknowledged

L-04: Payable Owner Functions on TradingSafe Trap ETH via No-Op Publish Hooks

Severity: Low	Impact: Low	Likelihood: Low
Files: src/trading-safe/TradingSafe.sol	Status: Fixed	

Description:

`MultiSig.setThreshold`/`configureOwners` and `RecoveryManager.recoverSafe`/`cancelRecovery` are `payable` so that on the OP-side `EtherFiSafe`, `msg.value` can cover the LayerZero fee for broadcasting the mutation to mainnet via `OwnerBridgePublisher`. `EtherFiSafeBase` declares the `_publishConfigureOwners`/`_publishSetThreshold`/`_publishRecover`/`_publishCancelRecovery` hooks these functions call as empty no-ops by default, and only `EtherFiSafe` overrides them to wire in `OwnerBridgePublisher`'s real logic — including its explicit `_refundFullValue()`/`_refundLeftover()` refund paths. `TradingSafe` (via `TradingOwnerBridgeReceiver` → `EtherFiSafeCore`) never overrides these hooks, so it inherits the bare no-op stubs. Since the four functions remain `payable` on `TradingSafe` too (they're defined once in the shared `MultiSig`/`RecoveryManager` mixins), any ETH sent alongside these calls on mainnet is accepted into the safe's balance but never refunded — there's no code path that returns it, unlike every skip/leftover case `OwnerBridgePublisher` explicitly handles for OP.

Recommendations:

Give the default no-op hooks in `EtherFiSafeBase` a refund fallback (forward the full `msg.value` back to `msg.sender`, mirroring `OwnerBridgePublisher._refundFullValue()`, alternatively reject if msg.value is not 0) instead of doing nothing, so any safe that doesn't override them — now or in the future — can't strand value.

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

L-05: Unauthenticated processTopUp/processTopUpFromContracts Let Anyone Front-Run and DOS redirectToTradingSafe

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/top-up/TopUp.sol	Status: Acknowledged	

Description:

`TopUpFactory.processTopUp` and `processTopUpFromContracts` are both fully public with no role check and no token-support validation. Anyone can call either with an arbitrary `topUp` address and an arbitrary token list — including a trading-supported-but-topup-unsupported token like Ondo SPY that's sitting in a user's `TopUp` waiting to be routed via `redirectToTradingSafe`. Since `TopUp.processTopUp` only checks `msg.sender == owner()` and the factory is always that owner when calling through these wrappers, the sweep goes through unconditionally, pulling the token's entire balance out of the `TopUp` and into the `TopUpFactory` contract itself.

Two consequences:

1) the legitimate `redirectToTradingSafe(topUp, token, amount)` call the backend later issues will revert — the `TopUp` no longer holds the balance, so `TopUp.redirectToTradingSafe`'s `safeTransfer` fails — permanently denying that specific recovery for anyone able to front-run it (which is anyone, since the sweep is permissionless); and

2) the swept funds now sit in the `TopUpFactory`'s own balance, where the only exit is the privileged `recoverFunds`, which sends to the generic `recoveryWallet` — not the user's `TradingSafe`.

Recommendations:

Alter `processTopUp`/`processTopUpFromContracts` (or push the check down into `TopUp.processTopUp` itself) to only sweep tokens that are actually topup-supported (`$.supportedTokens.contains(token)`), rejecting or skipping any token that isn't

Customer Response:

Acknowledged - "*The only expected side effect is that the backend redirect call may fail because the funds were already processed. We are comfortable with this behavior.*"

Fix Review:

Acknowledged

L-06: redirectToTopUp Doesn't Validate the Destination TopUp Is Deployed

Severity: Low	Impact: Low	Likelihood: Low
Files: src/trading-safe/TradingSafeFactory.sol	Status: Acknowledged	

Description:

`redirectToTopUp` trusts `$.topUpAddress[tradingSafe]` unconditionally and transfers directly to it, with no check that the address is non-zero or corresponds to an actual `TopUp` contract deployed by `TopUpFactory`. This mapping is only ever populated once, at `deployTradingSafe` time, from whatever value the `TRADING_SAFE_FACTORY_ADMIN_ROLE` caller supplied — there's no on-chain guarantee it stays correct or was ever validated as a real TopUp instance in the first place. Unlike the mirror path (`TopUpFactory.redirectToTradingSafe`, which explicitly reverts with `TradingSafeNotDeployed` if the resolved destination isn't a real, registered safe), a bad or stale entry here doesn't fail safely — it can move funds to an arbitrary address with no recovery path, since only genuine `TopUp` contracts expose `processTopUp`/`redirectToTradingSafe` to get funds back out.

Recommendations:

Before transferring, validate the resolved `topUp` address against `TopUpFactory`, mirroring the check already present on the other side.

Customer Response:

Acknowledged - "*This is expected behavior because TopUp contracts are deployed lazily only after enough funds have entered the safe.*"

Fix Review:

Acknowledged

L-07: Malicious owner can always cancel cross-chain recovery

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/trading-safe/TradingSafe.sol	Status: Fixed	

Description:

The destination TradingSafe is meant to be a bridge-driven mirror of the source safe, but it still inherits and exposes the native RecoveryManager/MultiSig functions — including `cancelRecovery`. Bridged recovery is timelocked: applyBridgeRecover stages the incoming owner with a future startTime, and until that time passes the original raw owner set stays fully in control on the destination.

So when legitimate parties attempt to recover from a compromised owner O — `recoverSafe(N, ...)` on the source, which bridges over and stages N with a future startTime — O is still an authoritative destination owner for the whole timelock window. O simply calls the native cancelRecovery on the destination and wipes the pending recovery. The source completes and evicts O there, but the destination recovery is unilaterally cancelled: O keeps full ownership of the TradingSafe and its assets while N controls the source. The source finalization is never bridged, so nothing later removes O, and O can cancel any future attempt. The emergency mechanism against a compromised owner is defeated on the destination.

Recommendations:

Make the destination bridge-driven only: disable the native owner/recovery mutation entry points on TradingSafe (cancelRecovery, recoverSafe, configureOwners, setThreshold) so owner state changes solely through authenticated applyBridge* calls. With no local cancelRecovery, a locally-present old owner cannot override a bridged recovery.

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

Informational Severity Issues

I-01: cancelSwap digest built inline instead of via a helper

Files:

src/enso/EnsoSwapModule.sol

Description:

The request path builds its digest through a dedicated `_requestDigest` helper, but `cancelSwap` assembles its digest inline with a live `useNonce()` embedded in the `abi.encodePacked`.

Recommendations:

Extract a `_cancelDigest(safe, nonce)` helper (reading the nonce in the caller, mirroring `requestSwap`) so both paths are symmetric and easier to read.

Customer Response:

Fixed in **#223** — commit `8569293`

Fix Review:

Fixed

I-02: Order struct comment omits recipient from the "not enforced on-chain" list

Files:

src/enso/EnsoSwapModule.sol

Description:

The comment lists `dstChainId`, `dstToken`, and `minOut` as carried for signed intent/events only and not enforced on-chain, but omits `recipient`, which is also not enforced on-chain.

Recommendations:

Update the comment to include `recipient` so it is clear that it is not enforced like the other parameters.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-03: TradingLens.getSafeData(safe, requestedTokens) doesn't dedupe the caller-supplied token list

Files:

src/trading-safe/TradingLens.sol

Description:

The two-arg `getSafeData(address safe, address[] calldata requestedTokens)` overload passes the caller-supplied array straight into `_getSafeData`, which loops over it and calls `_buildTokenInfo(safe, tokenList[i])` per index, summing each entry's `valueUsd` into `totalValueUsd`. Neither `_getSafeData` nor `_buildTokenInfo` checks for repeated addresses. If `requestedTokens` contains the same token N times, that token's balance/value is read and added N times, so `totalValueUsd` (and the returned `tokens` array) no longer reflects the safe's actual holdings — it's inflated by however many duplicates were passed.

Recommendations:

Consider checking for duplicate entries

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-05: isEnabled/getEnabledDestinations Don't Filter Against Global Destinations, Can Report Stale EIDs as Active

Files:

src/ownership-bridge/OwnershipBridgeSender.sol

Description:

If an admin removes a destination globally via `configureDestination(destEid, _, false)`, any safe that had previously been enabled for that `destEid` keeps it in its `enabledEids[safe]` set indefinitely unless someone explicitly calls `disable(safe, destEid)`. `isEnabled(safe, destEid)` will keep returning `true` for that removed destination, and `getEnabledDestinations(safe)` will keep including it in the returned array — even though `_dispatch`/`_quoteTotal` (via `_liveEnabledEids`) would silently skip it on any real publish, since it's no longer in the global set. This is only inconsistent at the read-API level (no funds or dispatch-correctness impact — the actual send/quote logic filters correctly), but it means anything relying on these two views for visibility — a front-end showing "bridging active to chain X," a keeper deciding whether to expect a message for a given destination, monitoring/alerting — gets a false picture of what will actually happen on the next publish.

Recommendations:

Either filter both views through the same live-intersection logic `_liveEnabledEids` already implements (renaming/exposing it, or adding `isLive(safe, destEid)`/`getLiveEnabledDestinations(safe)` variants) so callers can get an accurate picture without cross-referencing `getDestinations()` themselves, or clearly document that `isEnabled`/`getEnabledDestinations` reflect raw per-safe bookkeeping only and callers must intersect with `getDestinations()` to know what's actually live.

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

I-06: _quoteTotal Doesn't Mirror _dispatch's Peer Check

Files:

src/ownership-bridge/OwnershipBridgeSender.sol

Description:

`_dispatch` treats a live destination with no configured LZ peer as a hard failure for the whole publish call (`revert PeerNotConfigured(destEid)`) — since a `configureOwners`/`setThreshold`/`recover`/`cancelRecovery` publish sends to every live destination atomically, one missing peer means the entire operation can't go through. `_quoteTotal` — used by `quoteConfigureOwners`/`quoteSetThreshold`/`quoteRecover`/`quoteCancelRecovery` to let callers estimate `msg.value` before calling the real `publish*` — doesn't check `peers[destEid]` at all, so it can hand back a fee quote (or revert with an unrelated LZ-internal error) for a set of destinations that would actually fail atomically the moment `publish*` is called. That's a quote/dispatch inconsistency: a caller can get a seemingly valid `totalFee`, forward exactly that in `msg.value`, and still have the transaction revert on `PeerNotConfigured` rather than succeed.

Recommendations:

Add the same `peers[destEid] == bytes32(0)` check to `_quoteTotal`'s loop, but instead of reverting, short-circuit and return `0`

Customer Response:

Fixed in **#223** — commit `17d401c`

Fix Review:

Fixed

I-07: recoverFunds and redirectToTradingSafe Have Overlapping Authority Over Trading-Supported, TopUp-Unsupported Tokens

Files:

src/top-up/TopUp.sol

Description:

```
function recoverFunds(address token, uint256 amount) external nonReentrant
onlyRoleRegistryOwner {
    ...
    if ($.supportedTokens.contains(token)) revert OnlyUnsupportedTokens();   //
only guard on token
    if ($.recoveryWallet == address(0)) revert RecoveryWalletNotSet();
    ... transfer to $.recoveryWallet ...
}
function redirectToTradingSafe(address topUp, address token, uint256 amount)
external nonReentrant whenNotPaused {
    if (!roleRegistry().hasRole(TOPUP_FACTORY_REDIRECT_ROLE, msg.sender)) revert
OnlyRedirectRole();
    ...
    if ($.supportedTokens.contains(token)) revert OnlyUnsupportedTokens();   //
same guard
    ...
    if (!ITradingSafeFactory(tsFactory).isSupportedToken(token)) revert
TokenNotTradingSupported();  // additional guard
    ... transfer to user's TradingSafe ...
}
```

Both functions gate on the exact same base condition — `token` not in `$.supportedTokens` (i.e. not configured for TopUp's own bridge flow). `redirectToTradingSafe` narrows that further by also requiring the token be trading-supported, and sends it to the user's own TradingSafe. `recoverFunds` has no such carve-out — it accepts *any* token that merely isn't topup-supported, including ones that are trading-supported, and sends it to the protocol's generic `recoveryWallet` instead. So for the exact token class `redirectToTradingSafe`'s own docstring calls out as its purpose ("trading-supported, not-topup-supported tokens (e.g. Ondo SPY) that

landed at the TopUp address by mistake"), two independently privileged roles — `onlyRoleRegistryOwner` for `recoverFunds`, `TOPUP_FACTORY_REDIRECT_ROLE` for `redirectToTradingSafe` — each have full, uncoordinated authority to claim the same balance and route it to two different places. Whichever one acts first wins, with no on-chain signal that the token was "supposed to" go to the user's TradingSafe. If the recovery-wallet path fires first (whether by operator error, differing runbooks, or simply being unaware a redirect was pending), a user's trading-eligible assets end up in the protocol's general recovery wallet instead of their own safe, requiring an off-chain/manual process to make the user whole rather than following the intended automated path.

Recommendations:

Make sure to document the behavior if it is intentional

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-08: AcrossSwapModule deposit path leaves the spokePool approval unreset

Files:

src/across/AcrossSwapModule.sol

Description:

`_dispatchDeposit` approves the `spokePool` for `order.srcAmount` and calls `depositV3`, but never resets
the allowance back to zero afterward. The other dispatch paths — `_dispatchOriginSwap` and Enso's
`_dispatchSwap` both approve, call, then `approve(target, 0)`.

The concern is that this relies on an invariant that isn't locally enforced — that `spokePool` always pulls exactly `srcAmount`. `spokePool` is admin-settable and sits outside the user signature, so an upgraded or misconfigured pool that pulls less would leave a standing allowance to a general-purpose
bridge contract. If that ever happened, the next deposit of a USDT-style token would also revert on
`approve(spokePool, srcAmount)` (non-zero over non-zero), bricking further deposits for that token.

Recommendations:

Add a trailing `approve(spokePool, 0)` to `_dispatchDeposit` so it matches the other two paths.

Customer Response:

Fixed in **#223** — commit `29f84f6`

Fix Review:

Fixed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.